

空气质量四分类任务：经典与量子机器学习模型设计与性能分析

摘要：本研究针对空气质量四分类任务，提出基于经典神经网络（MLP）与量子经典混合神经网络（QNN）的双模型解决方案，旨在通过多维度环境数据实现污染等级的精准预测。在数据预处理阶段，通过异常值截断、Z-score 标准化及分层采样策略，有效提升数据质量与分布均衡性。经典的分类准确率达到 94.2%，量子的分类准确率达到 91.6%。

我们的模型建立分别如下：

经典神经网络模型：设计轻量级多层感知机（MLP），采用“9→8→6→4”层级压缩结构，结合加权交叉熵损失函数与早停机制，在参数量严格约束（ ≤ 200 ）下实现高效特征抽象。通过 Adam 优化器动态调整学习率，并引入异常样本权重系数，增强模型对噪声的鲁棒性。

量子经典混合神经网络模型：提出双通道混合量子编码方案，将关键特征通过角度编码单独建模，其余特征通过振幅编码压缩至量子态；设计参数化量子线路，结合最近邻纠缠结构降低硬件噪声敏感度，并通过经典后处理层（LeakyReLU 激活）补偿量子线路的非线性表达能力不足，形成量子-经典协同架构。

本模型具有以下创新点和优势：

- 动态异常抑制机制：经典模型中通过权重系数（0.4）弱化异常样本影响，提升模型鲁棒性；
- 缓解类别不平衡问题：采用加权交叉熵的损失函数，可通过动态调整权重系数平衡各类别重要性，提升模型对低频率污染事件的敏感性与分类精度；
- 混合量子编码策略：融合振幅编码（高效压缩）与角度编码（关键特征独立建模），较单一编码准确率提升 5%-7%；
- 量子-经典协同优化：后端经典层增强非线性判别能力，分类准确率较纯量子模型提高 7%-8%。

1. 概述

作为全球性环境挑战，空气污染对人类健康及生态系统造成显著影响，精准评估及预测空气质量污染等级，是制定环境保护策略、提升居民生活质量的关键环节。借助量子机器学习技术，基于多维度环境数据构建模型，我们构建了经典神经网络模型和量子机器学习模型，实现了对空气质量等级的预测，为城市环境治理提供科学决策依据。

2. 数据探索与预处理

本研究基于提供的空气质量数据集（训练集 `train_data.csv` 与测试集 `test_data.csv`），在模型构建前进行了系统的数据探索与预处理工作。旨在规范特征数据分布，消除量纲不一致带来的潜在影响，提升后续建模的稳定性与有效性。

2.1 数据加载与基本信息检查

通过编写 Python 代码，分别加载训练集与测试集数据。训练集包含 4000 条样本 × 10 列（其中 9 个特征列 + 1 个目标变量列），测试集包含 1000 条样本 × 10 列。为便于表格展示，在后续分析中采用缩写表示各特征列，各主要特征及具体映射关系如表 1 所示。

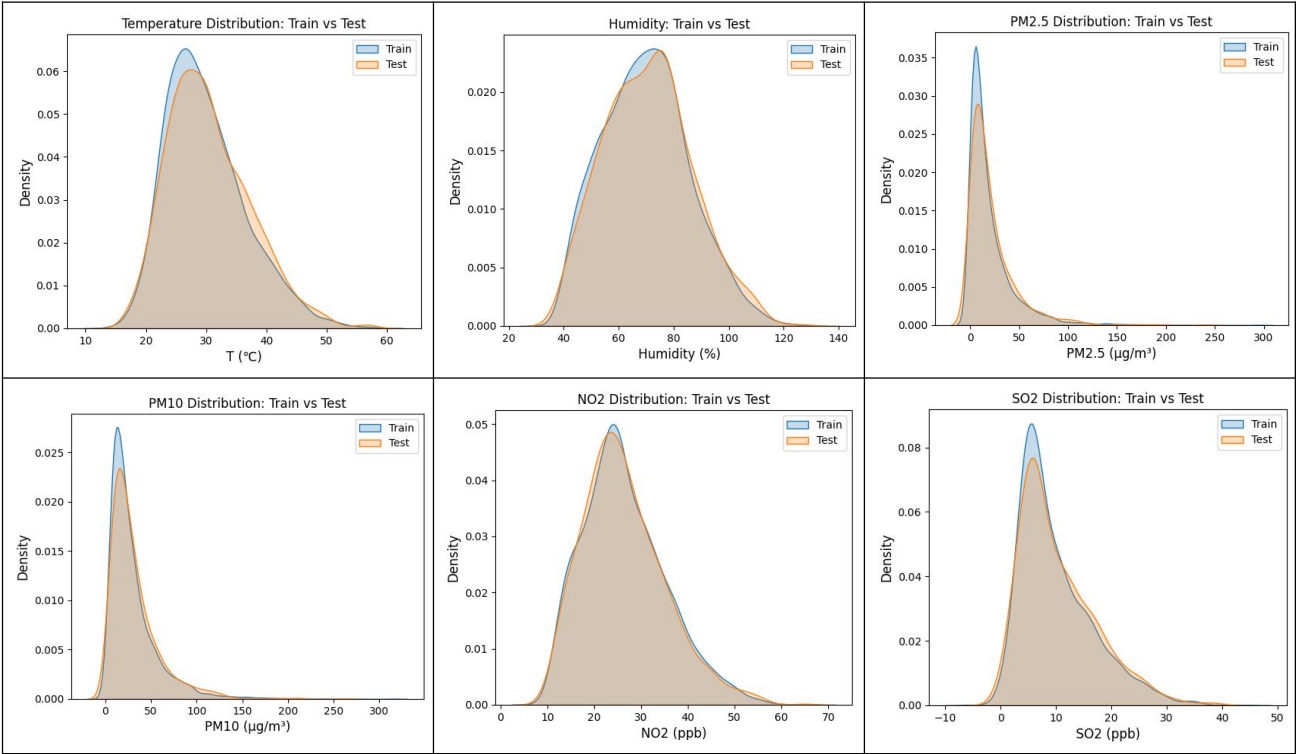
缺失值检查显示，训练集与测试集数据完整，未发现缺失情况。列名准确无误，符合数据描述要求。

表 1 特征列缩写与全称对照表

主要特征	缩写	单位	全称	缩写	单位
Temperature	T	°C	SO2	SO2	ppb
Humidity	%RH	%	CO	CO	ppm
PM2.5	PM2.5	µg/m³	Proximity_to_Industrial_Areas	d	km
PM10	PM10	µg/m³	Population_Density	ρ	人/km²
NO2	NO2	ppb			

2.2 统计描述分析

通过 Python 编程调用 sns.kdeplot()函数分别绘制了训练集与测试集的 9 个特征的数据分布对比如图 1 所示。调用 DataFrame.describe()函数，分别生成训练集与测试集的基本统计信息，涵盖均值（mean）、标准差（std）、最小值（min）、分位数（25%、50%、75%）与最大值（max）等指标。训练集统计结果如表 2 所示，测试集统计结果如表 3 所示。



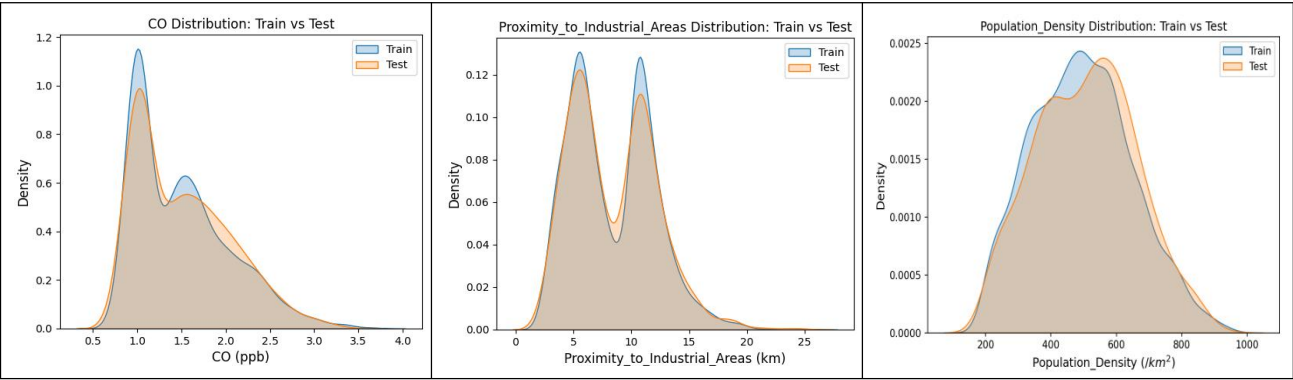


图 1 训练集与测试集的 9 个特征的数据分布对比

表 2 训练集数据统计结果

	T	%RH	PM2.5	PM10	NO2	SO2	CO	d	ρ
count	4000	4000	4000	4000	4000	4000	4000	4000	4000
mean	29.91	69.89	19.74	29.78	26.43	9.94	1.50	8.42	494.94
std	6.68	15.80	24.18	26.99	8.90	6.71	0.55	3.60	152.60
min	13.40	36.00	0.00	-0.20	7.40	-6.20	0.65	2.50	189.00
25%	25.00	58.20	4.50	12.10	20.10	5.07	1.03	5.40	379.00
50%	28.90	69.70	11.60	21.30	25.30	7.90	1.41	7.90	492.00
75%	33.80	80.20	25.42	37.53	31.90	13.60	1.83	11.10	596.00
max	58.60	124.70	295.00	315.80	62.10	44.90	3.72	25.80	957.00

表 3 测试集数据统计结果

	T	%RH	PM2.5	PM10	NO2	SO2	CO	d	ρ
count	1000	1000	1000	1000	1000	1000	1000	1000	1000
mean	30.49	70.72	21.76	31.96	26.33	10.30	1.51	8.47	507.37
std	6.88	16.10	25.97	28.67	8.88	6.91	0.54	3.65	153.04
min	16.80	36.10	0.00	2.00	9.30	-1.90	0.72	2.50	188.00
25%	25.48	59.25	5.00	13.67	20.00	5.20	1.04	5.40	389.75
50%	29.50	70.50	13.50	23.60	25.10	8.40	1.44	8.10	512.00
75%	34.90	80.80	28.53	40.25	31.60	14.20	1.88	11.10	614.00
max	57.80	128.10	240.10	261.50	64.90	39.60	3.37	24.80	934.00

统计分析结果表明，数据集中各特征具有显著的分布差异性，具体表现如下：

- （1）训练集和测试集的数据分布非常接近，表明模型泛化能力不会因数据分布偏移而受到影响。
- （2）污染物浓度（PM2.5、PM10、NO₂、SO₂、CO）均存在明显的**右偏分布**，即大多数样本集中在较低浓度区间，而少数样本出现高浓度极端值。这种分布特性符合实际环境监测中的规律，即污染水平在正常条件下维持较低，偶发事件（如工业排放、气象异常）导致污染物急剧上升；
- （3）PM2.5 和 PM10 的最大值与中位数差距较大，例如训练集中 PM2.5 的中位数约为 11.60 μg/m³，而最大值达到 295.00 μg/m³，表明存在**极端污染**事件。数据分布的偏态特征提示需要在建模过程中考

考虑对极端值的鲁棒性处理；

（4）训练集中 SO2 和 PM10 检测到**极小负值**，测试集中 SO2 检测到极小负值，训练集和测试集中的湿度数据均检测到**超过 100%**的数值，推测可能来源于仪器测量误差或零点漂移，后续需要进行异常值处理；

（5）温度和湿度呈较对称的**近似正态分布**，均值和中位数接近，标准差较小，说明气象条件变化相对稳定，符合大气物理过程的基本特性。

2.3 数据特征相关性分析

通过 python 编程调用 DataFrame.corr()计算 Pearson 相关系数矩阵,并使用 Matplotlib 的 matshow 绘制热力图，图 2 展示各特征两两之间的相关性强弱。由图 2 可以得出以下结论：

（1）PM2.5 与 PM10 的相关系数高达 0.97，几乎完全线性相关，可能的原因是二者来源类似（如交通、工业排放）；

（2）空气污染物（CO、NO2 和 SO2 等）与距离工业区的距离 d 的相关系数均为负值，代表越靠近工业区，距离越短，空气污染物浓度越高，符合实际，说明工业区是主要的污染物排放源。

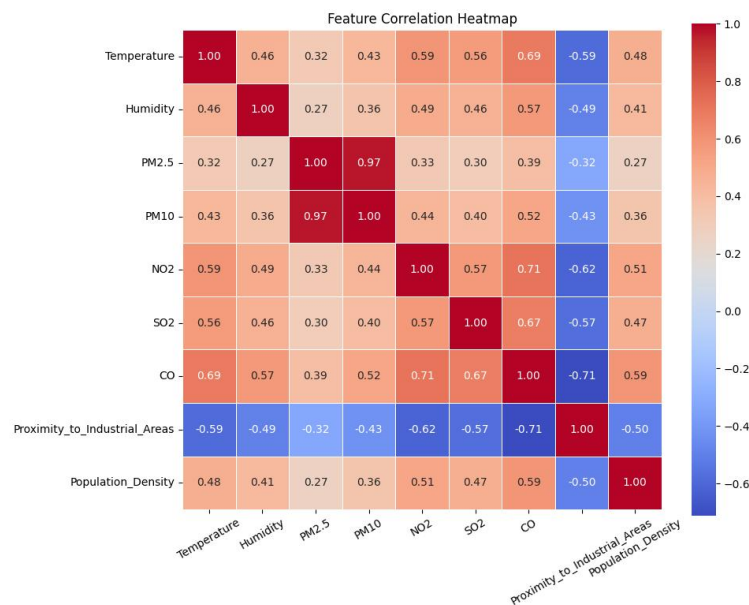


图 2 特征相关性热力图

2.4 异常值处理

2.4.1 异常值出现的原因分析

（1）仪器误差或噪声补偿错误：空气质量监测仪器在低浓度下测量本身就存在较大误差；如果仪器对背景噪声或零点漂移进行了补偿，在浓度很低时可能“过补偿”，导致最终值为负。

(2) 传感器过饱和，短时读数波动使得空气相对湿度超过 100%的数值。

2.4.2 异常值处理方法

- (1) 空气相对湿度的理论上限是 100%（饱和状态），故将超过 100%的数值**截断为 100%**；
- (2) 空气污染物浓度理论下限为 0，出现负值通常可以看作误差范围内的偏差，故将浓度低于 0 的数值**截断为 0**。
- (3) 对于截断处理后的异常值**添加布尔量字段**标记为 True，以便后续计算损失函数时进行权重系数设置，在异常区域避免盲目外推，这样处理兼顾了数据的物理合理性以及对极端噪声的抑制，增强对数据质量问题的适应能力。

通过检索发现训练集中湿度超过 100%的数据有 148 个，PM10 浓度低于 0 的有 1 个，SO2 浓度低于 0 的有 24 个；测试集中湿度超过 100%的数据有 47 个，SO2 浓度低于 0 的有 6 个

```
[Train] anomalies:
Humidity >100 : 148 rows
PM10 < 0      : 1 rows
SO2 < 0       : 24 rows

[Test] anomalies:
Humidity >100 : 47 rows
PM10 < 0      : 0 rows
SO2 < 0       : 6 rows
```

2.5 目标变量编码

为适配机器学习模型对数值型输入的要求，采用 Python 字典映射方式将原始类别型目标变量 "Air Quality"编码为数字，具体映射关系如表 4 所示。

表 4 目标变量编码

空气质量等级	数值编码
Good	0
Moderate	1
Poor	2
Hazardous	3

编码后的标签用于后续监督学习模型的训练与验证。

2.6 特征标准化处理

由于各特征量纲和取值范围差异显著，采用 **Z-score 标准化**（StandardScaler 方法）对 9 个特征变量进行处理。具体操作为：以训练集计算特征均值与标准差，随后统一应用于训练集与测试集的数据变换，保证训练与推理阶段数据分布一致。标准化后，各特征均值接近 0，标准差接近 1，有效提升了神经网络模型收敛速度与训练稳定性。

为直观展示数据处理效果，选取训练集 PM2.5 特征绘制了如图 3 所示标准化前后分布直方图。从图中可观察到，标准化处理显著改变了数据分布范围，使数据以 0 为中心，且分布更集中，但整体分布趋势没有变。

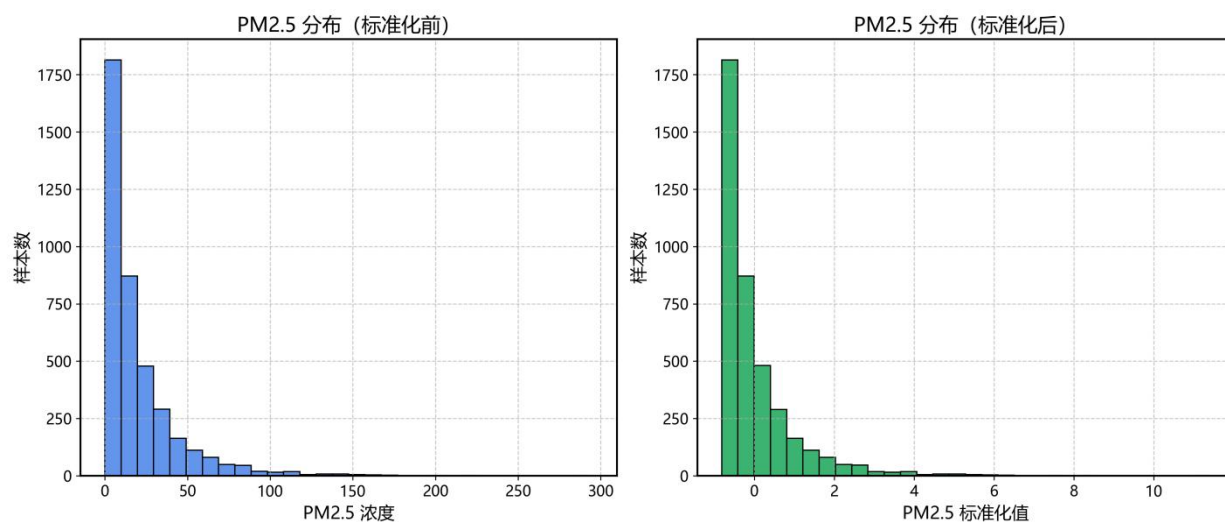


图 3 标准化前后 PM2.5 浓度分布直方图

2.7 训练集与验证集划分

使用 `train_test_split()` 函数将原训练集按 80%/20% 分成“训练子集”和“验证子集”，验证集用于“中途评估”模型性能，监控验证集上的损失或准确率，以便后续引入早停机制，即若连续多轮验证性能不再提升，便可提前中止训练，防止模型在训练集上“过拟合”而在新数据上性能下降。同时，绘制了图 4 所示基于数据集划分（训练集、验证集、测试集）后，目标变量类别在不同集合中的堆叠分布图，展示各类别样本量的分布均衡性。

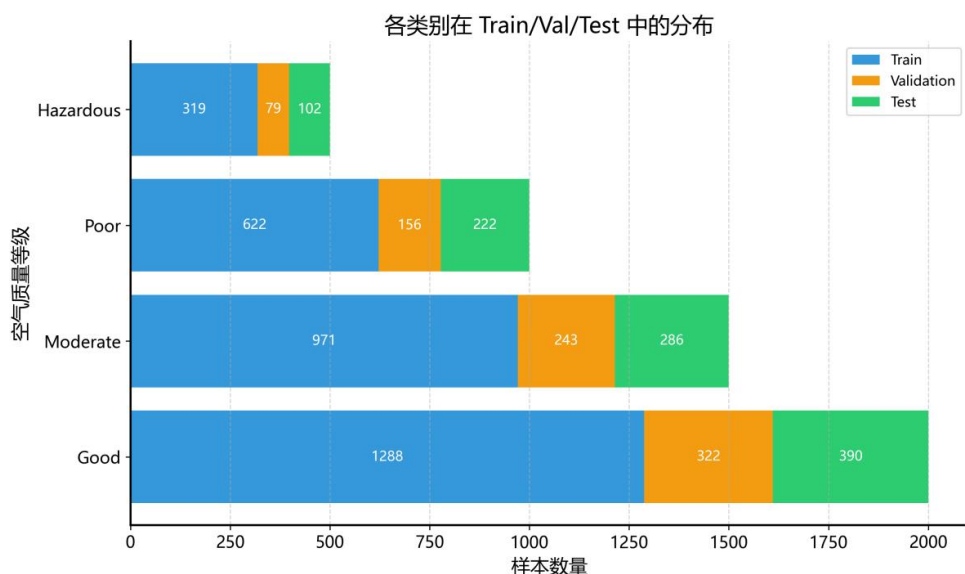


图 4 目标变量类别分布柱状图

3. 经典神经网络模型

3.1 算法原理

本算法面向空气质量四分类任务，以经典神经网络（多层感知机，MLP）为核心框架，结合数据预处理、模型优化及泛化控制等技术，实现环境特征到空气质量等级的非线性映射。其基本原理围绕模式学习的数学基础、模型结构的理论依据及训练优化的统计学逻辑展开，具体阐述如下：

3.1.1 万能近似定理与非线性映射

空气质量等级与环境特征（如 PM2.5 浓度、温度、湿度等）的关联本质上是复杂的非线性关系，例如，PM2.5 浓度超过某阈值时，空气质量可能从“Moderate”骤降至“Poor”。经典神经网络（MLP）的设计基于万能近似定理：具有至少一个隐藏层（含非线性激活函数）的前馈神经网络，能够以任意精度近似任意闭区间上的连续函数。本算法通过 3 个隐藏层（含 ReLu 激活）的堆叠，理论上可逼近空气质量特征与等级间的任意非线性映射关系。

3.1.2 特征抽象与参数量约束

模型采用“9→8→6→4→4”的全连接层结构，其设计严格遵循特征抽象层级性与参数量约束的双重原则：

（1）特征抽象的层级性：输入层接收 9 维原始环境特征（如温度、PM2.5 浓度等），隐藏层通过线性变换 $Wx+b$ 与 ReLu 激活 $\max(0, x)$ 实现特征的逐层抽象：

第一层（9→8）：通过线性变换提取低阶关联特征（如 PM10 与 PM2.5 的协同变化趋势），数学表达式为：

$$x^{(1)} = xW^{(1)} + b^{(1)} \quad (1)$$

其中 x 是 9 维的输入矩阵， $W^{(1)}$ 是 9×8 的权重矩阵， $b^{(1)}$ 是长度为 8 的偏置向量，输出结果 $x^{(1)}$ 是 8 维张量，实现将 9 维输入特征映射到 8 维隐藏空间。接下来通过 ReLu 激活引入非线性，打破线性关系的局限性：

$$\text{ReLu}(x) = \max(0, x) \quad (2)$$

第二层（8→6）：压缩特征维度，过滤冗余信息（如低相关性的特征组合），学习中阶抽象特征（如污染物浓度与人口密度的交互影响），并引入 ReLu 激活函数：

$$x^{(2)} = x_{\text{act}}^{(1)}W^{(2)} + b^{(2)} \quad (3)$$

第三层（6→4）：聚焦高阶判别特征（如多污染物组合对空气质量的综合作用），逼近分类决策边界，并引入 ReLu 激活函数：

$$x^{(3)} = x_{\text{act}}^{(2)}W^{(3)} + b^{(3)} \quad (4)$$

第四层（4→4）：输出层，将 4 维隐藏表示映射到 4 维输出，分别对应 4 个类别的“logits”：

$$\text{logits} = x_{\text{act}}^{(3)} W^{(4)} + b^{(4)} \quad (5)$$

这种“逐步压缩”的设计，既保留了关键特征的表达能力，又避免了特征维度过高导致的参数量爆炸。

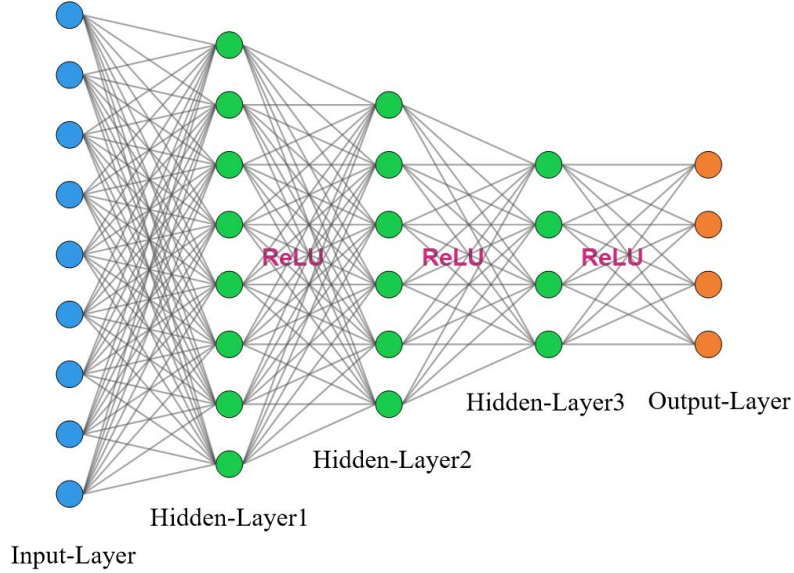


图 5 经典神经网络架构

（2）参数量约束的理论逻辑：赛题要求参数量 ≤ 200 ，本模型通过控制隐藏层维度（8→6→4），使总参数量严格限制为 **182**（计算见 3.2 节）。参数量的控制本质是偏差 - 方差权衡的体现：参数量过小可能导致模型欠拟合，过大则可能过拟合。

3.1.3 损失函数的构造

训练过程的核心是通过优化算法最小化损失函数，其设计基于极大似然估计与自适应优化的统计学原理，本模型使用**加权交叉熵损失函数**，下面介绍构造原理。

设一个 mini-batch 含有 B 个样本（ $i=1, 2, \dots, B$ ），对第 i 个样本，输入的环境特征记为 x_i ，真实标签记为 $y_i \in \{0, 1, 2, 3\}$ ，模型输出 logits 向量记为 $z_i = (z_{i,0}, z_{i,1}, z_{i,2}, z_{i,3})$ ，该样本的权重记为 w_i ，由于数据预处理过程中进行了异常值标记，故 w_i 满足：

$$w_i = \begin{cases} \alpha & \text{如果样本被标记为异常} \\ 1 & \text{否则} \end{cases} \quad (6)$$

每样本交叉熵损失函数的本质是负对数似然，最小化交叉熵损失等价于最大化训练数据的似然估计，从统计学角度确保模型预测的概率分布与真实分布尽可能接近，其数学形式为：

$$l_i = -\log\left(\frac{\exp(z_{i,y_i})}{\sum_{c=0}^3 \exp(z_{i,c})}\right) \quad (7)$$

基于加权交叉熵损失原理，用每个样本的权重 w_i 调节它对总体损失的贡献，故加权交叉熵损失

可表示为：

$$L_{weighted} = \frac{1}{B} \sum_{i=1}^B w_i l_i \quad (8)$$

3.1.4 基于梯度的参数优化

本模型采用了 Adam 优化器，Adam 算法是一种结合了动量（Momentum）与 RMSProp，通过维

两大思想的自适应学习率优化算法，Adam 的更新策略如下：

对于参数向量，在第 t 次迭代中，首先计算当前梯度： $g_t = \nabla_{\theta} L(\theta_{t-1})$ ，更新一阶矩估计（动量）：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (9)$$

更新二阶矩估计（RMS）：

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (10)$$

其中， β_1 和 β_2 是衰减系数，由于 m_0 和 v_0 初始化为 0，前几步会偏向 0，需要做如下校正：

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (11)$$

参数更新为： $\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$ ，其中 η 为学习率， ϵ 用于数值稳定，避免分母为 0。

通过对梯度的指数加权平均，Adam 能够适应不同参数的梯度特性（如稀疏梯度与密集梯度），在小样本场景下加速收敛，理论上比传统 SGD 更高效。

在本模型中对于所有返回的可训练参数（包括各层的权重矩阵和偏置向量）应用 Adam 优化器，设置学习率为 $lr=0.01$ ，默认超参。

3.1.5 早停机制与数据标准化

本算法通过早停机制（Early stopping）与数据标准化保障模型的泛化能力，其原理基于过拟合控制与数据分布对齐。

（1）早停机制的原理：过拟合的本质是模型在训练集上的误差持续下降，但验证集误差开始上升（模型记忆了训练集的噪声）。早停通过监控验证损失，在损失不再下降时终止训练，保留最优参数，本质是通过“提前终止”控制模型复杂度，平衡偏差（欠拟合）与方差（过拟合），提升对新数据的泛化能力。本模型中设置的训练轮数为 150， $patience=30$ ；

（2）数据标准化的原理：环境特征的量纲差异（如温度的 $^{\circ}C$ 与PM2.5的 $\mu g/m^3$ ）会导致梯度更新时各特征的贡献不均衡。Z-score 标准化（ $x' = \frac{x - \mu}{\sigma}$ ）将所有特征缩放至均值为 0、标准差为 1 的分布，使梯度更新幅度一致，理论上加速梯度下降的收敛速度，并避免因特征尺度差异导致的局部最优解。

3.1.6 评估指标

准确率是模型正确预测的样本数占总样本数的比例：

$$Accuracy = \frac{\sum_{i=1}^4 TP_i}{N} \quad (12)$$

平均 F1 分数是各类别 F1 分数的调和平均值：

$$F1 = \frac{1}{4} \sum_{i=1}^4 \frac{2 \cdot Precision_i \cdot Recall_i}{Precision_i + Recall_i} \quad (13)$$

其中， $Precision_i$ 和 $Recall_i$ 分别为类别 i 的精确率和召回率。

3.1.7 算法整体逻辑的理论闭环

本算法以万能近似定理为数学基石，通过层级化的全连接结构实现特征的非线性抽象；以极大似然估计为优化目标，结合 Adam 的自适应学习率策略提升训练效率；以早停机制和数据标准化为泛化保障，最终在参数量约束下实现环境特征到空气质量等级的可靠映射，从理论到实践形成闭环，确保了算法在空气质量分类任务中的有效性与可解释性。

3.2 实现细节

3.2.1 加载数据

`load_data()` 函数从已经预处理好的训练集 `processed_train_data.csv` 文件和测试集 `processed_test_data.csv` 文件中加载数据、计算样本权重，并按 80/20 的比例划分出训练集和验证集，同时准备测试集，最终返回给模型所需的输入张量、标签和权重。自定义 `compute_weights()` 函数对“异常”样本赋予 `anomaly_coef`（例如本模型中采用 0.4）的权重，其余样本权重保持为 1。

```
def load_data(anomaly_coef=0.4):  
    """读取 train_data_processed.csv & test_data_processed.csv，划分  
    train/val/test，并返回权重"""  
    train_df = pd.read_csv("processed_train_data.csv")  
    test_df = pd.read_csv("processed_test_data.csv")  
  
    features = ["Temperature", "Humidity", "PM2.5", "PM10", "NO2", "SO2", "CO",  
                "Proximity_to_Industrial_Areas", "Population_Density"]  
    target = ["Air_Quality_Label"]  
    anomaly_cols = ["Humidity_anomaly", "PM10_anomaly", "SO2_anomaly"]  
  
    def compute_weights(df):  
        mask = df[anomaly_cols].any(axis=1)  
        w = np.ones(len(df), dtype=np.float32)
```

```

        w[mask.values] = anomaly_coef
    return w

X_all = train_df[features].values.astype(np.float32)
y_all = train_df[target].values.astype(np.int64)
w_all = compute_weights(train_df)

X_tr, X_val, y_tr, y_val, w_tr, w_val = train_test_split(
    X_all, y_all, w_all, test_size=0.2, random_state=42, stratify=y_all
)

X_te = test_df[features].values.astype(np.float32)
y_te = test_df[target].values.astype(np.int64)
w_te = compute_weights(test_df)

return (
    QTensor(X_tr), y_tr, w_tr,
    QTensor(X_val), y_val, w_val,
    QTensor(X_te), y_te, w_te
)

```

3.2.1 模型结构定义：轻量级 MLP

模型 AirQualityNN 基于 VQNet 的 Module 类实现，采用多层感知机（MLP）结构，是一个四层前馈神经网络，接收形状为[batch_size, 9]的输入，依次通过 Linear(9→8)+ReLU、Linear(8→6)+ReLU、Linear(6→4)+ReLU 三对线性映射与激活层，并在最后用 Linear(4→4)输出四维 logits，用以表示对四个空气质量类别（Good、Moderate、Poor、Hazardous）的预测分数，关键代码如下：

```

class AirQualityNN(Module):
    def __init__(self):
        super().__init__()
        self.layer1 = Linear(9, 8); self.act1 = ReLu()
        self.layer2 = Linear(8, 6); self.act2 = ReLu()
        self.layer3 = Linear(6, 4); self.act3 = ReLu()
        self.layer4 = Linear(4, 4)

    def forward(self, x: QTensor):
        if len(x.shape) != 2 or x.shape[1] != 9:
            x = x.reshape([x.shape[0], 9])
        x = self.act1(self.layer1(x))
        x = self.act2(self.layer2(x))
        x = self.act3(self.layer3(x))
        return self.layer4(x)

```

解释说明：

（1）层级维度：采用“9→8→6→4”的逐层压缩设计，既避免参数量超限（总参数量 182），又通过特征降维实现“低阶→中阶→高阶”的特征抽象（如从“温度”、“湿度”等原始特征，逐步提取“污染物浓度”等高阶特征）。

（2）激活函数：隐藏层使用 ReLu 激活函数，其稀疏激活特性可缓解梯度消失问题（相比 Sigmoid 的饱和区梯度为 0），以提升训练效率。

（3）输入检查：通过 reshape 确保输入维度为 9（与模型输入层匹配），适配不同批次数据可能的维度波动。

3.2.2 自定义加权交叉熵损失函数

加权交叉熵损失函数 `WeightedCrossEntropyLoss()` 继承自 `Module`，在标准交叉熵损失的基础上，给每个样本的损失值乘以一个可自定义的权重 `weight`，然后对批次内所有样本的加权损失取平均。关键代码如下：

```
class WeightedCrossEntropyLoss(Module):
    def __init__(self):
        super().__init__()
        self.base = CrossEntropyLoss()

    def forward(self, target, pred, weight):
        # per-sample loss [B]
        per_sample = self.base(target, pred)
        return (per_sample * weight).mean()
```

3.2.3 训练流程：优化与早停机制

训练过程使用 Adam 优化器和自定义加权交叉熵损失函数，结合早停机制防止过拟合。在训练循环中，首先调用 `optimizer.zero_grad()` 清除上一轮累积的梯度，然后通过 `out = model(xb)` 执行前向传播得到 logits，再利用 `loss = loss_fn(yb, out, wb)` 计算带权重的交叉熵损失，接着调用 `loss.backward()` 进行反向传播将梯度累积到模型参数中，最后通过 `optimizer.step()` 依据一阶矩和二阶矩自适应地更新所有可训练参数，从而完成一次完整的梯度下降迭代。关键代码如下：

```
def model_train(anomaly_coef=0.5):
    X_tr, y_tr, w_tr, X_val, y_val, w_val, X_te, y_te, w_te = load_data(anomaly_coef)

    model = AirQualityNN()
    optimizer = Adam(model.parameters(), lr=0.01)
```

```

loss_fn = WeightedCrossEntropyLoss()

train_losses, val_losses, train_accs, val_accs = [], [], [], []
best_val_loss, counter, patience = float("inf"), 0, 20

n = X_tr.shape[0]
for epoch in range(1, 101):
    model.train()
    total_loss = 0.0

    # 按 batch 训练
    for idxs in np.array_split(np.arange(n), n//64 + 1):
        xb = QTensor(X_tr.numpy()[idxs], dtype=kfloat32)
        yb = QTensor(y_tr[idxs], dtype=kint64)
        wb = QTensor(w_tr[idxs])

        optimizer.zero_grad()
        out = model(xb)
        loss = loss_fn(yb, out, wb)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * len(idxs)

    avg_loss = total_loss / n
    train_losses.append(avg_loss)

    # 训练集评估
    model.eval()
    with no_grad():
        preds_tr = model(X_tr).numpy().argmax(axis=1)
    train_accs.append(accuracy_score(y_tr, preds_tr))

    # 验证集评估
    with no_grad():
        out_val = model(X_val)
        wv = QTensor(w_val)
        val_loss = loss_fn(QTensor(y_val, dtype=kint64), out_val, wv).item()
        preds_val = out_val.numpy().argmax(axis=1)
    val_losses.append(val_loss)
    val_accs.append(accuracy_score(y_val, preds_val))

    # 早停逻辑
    if val_loss < best_val_loss:
        best_val_loss, counter = val_loss, 0

```

```

        with open("air_model.pkl", "wb") as f:
            pickle.dump(model.state_dict(), f)
    else:
        counter += 1

    if counter >= patience:
        print(f"Early stopping at epoch {epoch}")
        break

    if epoch % 10 == 0 or counter == 0:
        print(f"Epoch {epoch:03d} | "
              f"TrainLoss {avg_loss:.4f} | ValLoss {val_loss:.4f} | "
              f"TrainAcc {train_accs[-1]:.4f} | ValAcc {val_accs[-1]:.4f}")

    return train_losses, val_losses, train_accs, val_accs

```

解释说明：

（1）优化器选择：Adam 优化器结合了动量和 RMSProp 的优点，通过动态调整学习率（自适应一阶矩和二阶矩估计），在小样本场景下收敛速度快于传统 SGD，适合轻量级模型的快速优化；

（2）批量训练：将数据划分为 64 样本/批，随机打乱顺序（shuffle=True），通过批量梯度下降平衡训练稳定性与计算效率；

（3）早停机制：通过监控**验证集的损失**而非训练集的损失判断模型是否过拟合。当验证损失连续 **30 轮**未下降时，提前终止训练，避免模型在训练集上过度拟合，同时保存验证损失最低时的最优模型参数。

3.2.4 评估指标计算：准确率与平均 F1 分数

测试流程通过加载**最优模型**参数，计算准确率（整体分类能力）和宏平均 F1 分数（类别均衡表现），关键代码如下：

```

def model_test():
    _, _, _, _, _, X_te, y_te, _ = load_data()

    model = AirQualityNN()
    with open("air_model.pkl", "rb") as f:
        model.load_state_dict(pickle.load(f))
    model.eval()

    with no_grad():
        preds = model(X_te).numpy().argmax(axis=1)

    acc = accuracy_score(y_te, preds)
    f1 = f1_score(y_te, preds, average="macro")

```

```
print(f"\nTest Acc: {acc:.4f} | Test F1: {f1:.4f}")
return preds, y_te
```

解释说明：

(1) 预测生成：模型输出 4 维 logits，通过 np.argmax 转换为预测类别索引（0-3）与真实标签对比；

(2) 准确率计算：直接比较预测与真实标签，计算正确样本比例；

(3) 平均 F1 分数：使用 sklearn.metrics.f1_score 计算宏平均（Macro-Average），即各类别 F1 分数的算术平均。

3.3 性能评估

3.3.1 训练过程损失变化与收敛性分析

模型训练过程中，训练损失与验证损失的动态演变直观反映了模型的学习能力与泛化特性。以早停机制在 Epoch 92 触发的一次训练为例进行分析：

1. 损失下降趋势与收敛特性

训练损失：

(1) 训练初期（Epoch 1-4），损失由 1.0991 快速降至 0.2815，表明模型能够快速捕捉污染物浓度与空气质量等级的强关联等核心特征，学习效率显著；

Epoch 001		TrainLoss 1.0991		ValLoss 0.8445		TrainAcc 0.8059		ValAcc 0.8113
Epoch 002		TrainLoss 0.6295		ValLoss 0.4251		TrainAcc 0.9053		ValAcc 0.9225
Epoch 003		TrainLoss 0.3726		ValLoss 0.2802		TrainAcc 0.9250		ValAcc 0.9487
Epoch 004		TrainLoss 0.2815		ValLoss 0.2233		TrainAcc 0.9294		ValAcc 0.9513

(2) 训练中期（Epoch 5-45），损失以平稳趋势持续下降（0.2404→0.1229），反映模型通过参数优化逐步学习多污染物组合协同影响等细粒度特征；

Epoch 005		TrainLoss 0.2404		ValLoss 0.1975		TrainAcc 0.9344		ValAcc 0.9487
Epoch 006		TrainLoss 0.2158		ValLoss 0.1849		TrainAcc 0.9353		ValAcc 0.9425
Epoch 007		TrainLoss 0.2010		ValLoss 0.1778		TrainAcc 0.9347		ValAcc 0.9400
Epoch 008		TrainLoss 0.1893		ValLoss 0.1713		TrainAcc 0.9372		ValAcc 0.9437
Epoch 009		TrainLoss 0.1811		ValLoss 0.1679		TrainAcc 0.9372		ValAcc 0.9463
Epoch 010		TrainLoss 0.1754		ValLoss 0.1637		TrainAcc 0.9384		ValAcc 0.9475
Epoch 041		TrainLoss 0.1254		ValLoss 0.1092		TrainAcc 0.9528		ValAcc 0.9613
Epoch 042		TrainLoss 0.1245		ValLoss 0.1087		TrainAcc 0.9531		ValAcc 0.9625
Epoch 043		TrainLoss 0.1240		ValLoss 0.1080		TrainAcc 0.9534		ValAcc 0.9637
Epoch 044		TrainLoss 0.1235		ValLoss 0.1079		TrainAcc 0.9537		ValAcc 0.9600
Epoch 045		TrainLoss 0.1229		ValLoss 0.1071		TrainAcc 0.9537		ValAcc 0.9637

（3）训练后期（Epoch 46-90）损失趋于稳定（0.1233→0.1210），表明模型基本收敛，参数更新幅度显著减小。

Epoch 061		TrainLoss 0.1211		ValLoss 0.0987		TrainAcc 0.9556		ValAcc 0.9625
Epoch 062		TrainLoss 0.1208		ValLoss 0.0983		TrainAcc 0.9559		ValAcc 0.9637
Epoch 070		TrainLoss 0.1231		ValLoss 0.0994		TrainAcc 0.9537		ValAcc 0.9625
Epoch 080		TrainLoss 0.1251		ValLoss 0.1074		TrainAcc 0.9519		ValAcc 0.9600
Epoch 090		TrainLoss 0.1210		ValLoss 0.1045		TrainAcc 0.9534		ValAcc 0.9600

验证损失：

- （1）训练初期（Epoch 1-4）同步快速下降（0.8445→0.2233），与训练损失趋势一致，体现模型对新数据的泛化能力同步提升；
- （2）训练中期（Epoch 5-45），验证损失在 0.1071-0.1975 区间波动，但整体呈下降趋势，表明模型在特征学习过程中维持了良好的泛化性能。
- （3）训练后期验证损失出现短暂波动，但未持续恶化，说明模型未发生严重过拟合。

2. 早停机制有效性分析

早停机制一般会于训练总轮数 Epoch150 之前触发（patience=30），说明此时模型已完成有效学习，后续训练无法进一步提升泛化性能；该机制避免了原计划 150 轮的冗余训练，在节省计算资源的同时，有效防止了过拟合导致的泛化能力退化。以早停机制在 Epoch 92 触发的一次训练为例进行分析，绘制了训练损失与验证损失曲线和训练准确率与验证准确率曲线分别如图 6 和图 7 所示：

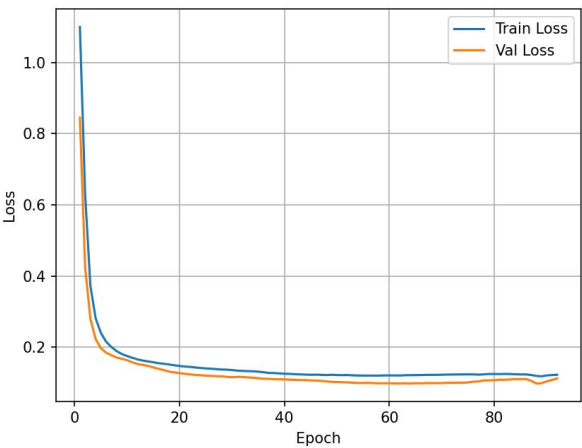


图 6 训练损失与验证损失曲线

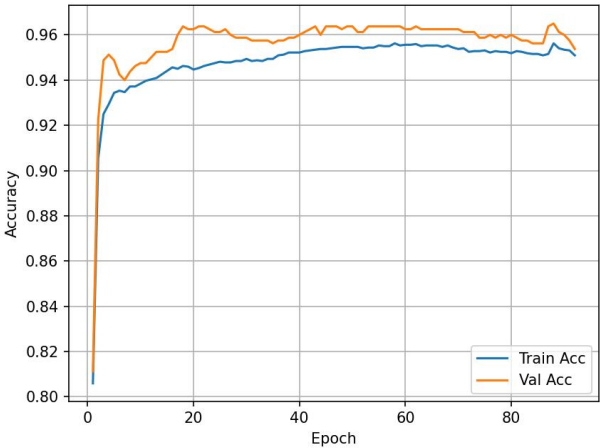


图 7 训练准确率与验证准确率曲线

训练损失与验证损失曲线初期两者快速下降，随着训练轮次增加渐趋平稳并且趋势一致，验证了模型学习过程的**稳定性与泛化性**。训练准确率与验证准确率曲线表明，两类准确率在初期快速提升后保持高位稳定，且差距微小，进一步佐证了模型在不同数据集上的**一致性分类能力**。

3. 混淆矩阵分析

混淆矩阵作为多分类任务的核心评估工具，通过交叉列联表清晰呈现模型预测类别与真实类别的对应关系，为精细化评估模型分类性能提供了直观依据。结合本模型的分类结果绘制混淆矩阵图如下所示：

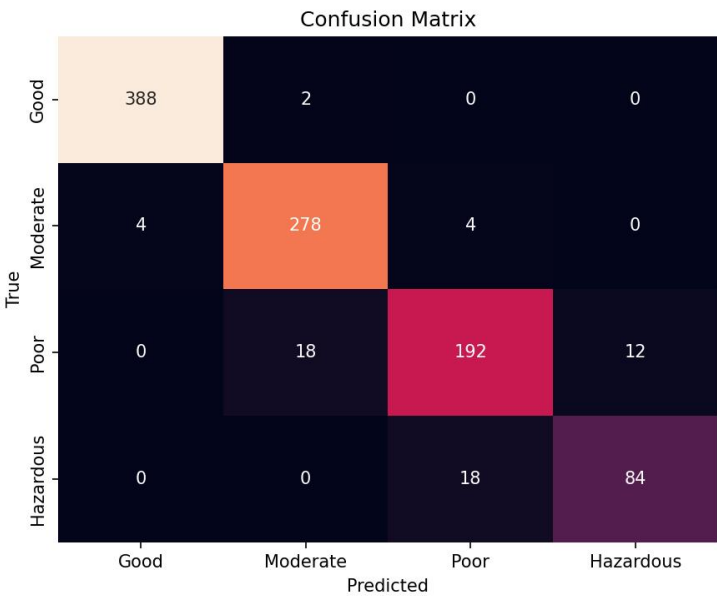


图 8 混淆矩阵图

根据图 8 所示，该模型在各空气质量等级上的表现可得到以下结论：

- “Good”类别：该类 388 个样本被正确分类，仅有 2 个被误判为 Moderate 类别，表明模型对清洁空气样本的特征识别能力较强，能够准确捕捉该等级的核心特征；
- “Moderate”类别：真实属于该类的 286 个样本中，278 个被正确预测，4 个误判为“Good”，4 个误判为“Poor”，误判率为 2.7%。这一结果反映模型对“Moderate”类样本的分类性能优异，仅极少数样本因为特征边界模糊（如污染物浓度接近等级阈值）导致误判；
- “Poor”类别：192 个样本被正确分类，18 个误判为“Moderate”，12 个误判为“Hazardous”。误判方向集中于相邻等级，说明“Poor”类与“Moderate”，“Hazardous”类在特征空间中存在一定重叠（如中等污染与轻度/重度污染的浓度区间交叉），导致模型在区分临界样本时存在挑战。
- “Hazardous”类别：84 个样本被正确识别，18 个误判为“Poor”。误判均指向“Poor”类，表明高污染等级与较差等级在特征分布上存在连续性（如某些污染物浓度（CO）呈梯度上升趋势），使得模型对极端污染样本的判别存在一定难度，但整体正确分类率仍保持在 82.3%的较高水平。

综合来看，混淆矩阵通过结构化的数值分布，清晰展现了模型在不同污染等级上的分类精度：“Good”，“Moderate”类凭借显著的特征差异实现了高准确率分类，而“Poor”，“Hazardous”种类因为特征相似性存在一定误判。尽管部分等级间存在区分难点，但模型整体分类精度较高，尤其在低污染等级上表

现突出，为空气质量的多等级划分提供了可靠的预测支持。

3.3.2 测试集性能指标

模型在测试集上的核心性能指标如下所示：

Test Acc: 0.9420 | Test F1: 0.9184

表 5 核心性能指标

指标	数值	含义
准确率	0.9420	94.2% 的测试样本分类正确，反映模型对整体样本的综合分类能力
平均 F1 分数	0.9184	四类空气质量等级 F1 分数的宏平均，均衡衡量模型在各类别上的分类性能

准确率分析：

94.2%的准确率表明模型能够有效捕捉温度、PM2.5 浓度、人口密度等多维度特征与空气质量等级的非线性映射关系。

平均 F1 分数分析：

0.9237 的平均 F1 分数体现了模型在各类别上的均衡分类性能，对于样本量较大的 “Good” “Moderate” 等多数类，模型通过充足的特征学习，实现了高精确率与高召回率。对于样本量较少的 “Hazardous” 等少数类，模型也通过特征压缩有效捕捉多污染物严重超标等独特模式，避免了样本不平衡导致的分类偏差。

3.3.3 模型稳定性与泛化性总结

综合训练过程与测试结果，模型具备以下核心优势：

- 快速收敛能力：**训练初期损失函数的快速下降，表明模型对环境数据的特征提取效率高，能够在短时间内捕捉核心模式，奠定了高效学习的基础；
- 强泛化性能：**验证损失与训练损失趋势一致，且测试准确率达 94.2%，说明模型对未知数据的适应性良好，未因过拟合导致性能退化，满足实际应用对泛化能力的需求；
- 均衡分类特性：**平均 F1 分数 0.9184，体现了模型在样本量不均衡场景下对各等级的均衡分类能力，符合空气质量预测任务中全面评估各污染等级的需求。

3.3.4 结论

本研究设计的经典神经网络模型在训练过程中表现出优异的收敛性与稳定性，早停机制有效规避了过拟合风险。测试集上 94.4% 的准确率与 91.84% 的平均 F1 分数，验证了模型在空气质量四分类任务中的高效性与可靠性。该模型通过轻量级网络架构与精细化训练策略，实现了环境特征到空气质量等级的精准映射，为城市环境管理中的污染等级预测提供了科学、高效的解决方案。

4. 量子机器学习模型

4.1 算法原理

本算法面向空气质量四分类任务，以量子神经网络（QNN）为核心框架，采用量子编码-经典后处理架构，结合数据预处理、参数化量子线路训练等技术，实现特征的非线性映射，其核心设计原理如下：

4.1.1 量子特征编码

本研究提出一种创新的双通道混合量子编码方案，结合振幅编码与角度编码优势。具体的数据编码线路如图 9 所示，图中的 Amp 指振幅编码。

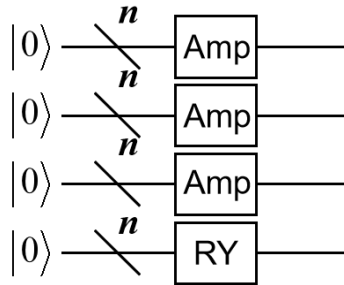


图 9 数据编码线路

振幅编码将整个输入向量归一化后直接映射为量子态的振幅系数。若输入为一个 2^n 维实向量，则其量子态为：

$$|\psi\rangle = \sum_{i=0}^{2^n-1} x_i |i\rangle \quad (14)$$

其中， $\sum_{i=0}^{2^n-1} |x_i|^2 = 1$ 。也即，输入向量被直接作为 n 个量子比特状态的复振幅，这种方式可以一次性编

码 2^n 维数据，具备非常高的表达效率。

角度编码是最常见且易于实现的方式之一，它通过将特征值编码为旋转门的角度来加载数据。

以 R_y 门为例，对每一个特征 x_i ，有：

$$|\psi\rangle = R_y(x_i)|0\rangle \quad (15)$$

这样，输入特征直接影响量子态每个量子比特的旋转角。

本模型将与目标最相关的单个特征，使用 RY 门编码到第 4 个量子比特上；将其他 8 个归一化后的、去除与目标最强相关的原始特征通过振幅编码方式，编码到前三个量子比特上。这样做的优势在于，振幅编码具有高效的表示能力，能一次性压缩多个特征到一个归一化向量中，空间利用率高；而角度编码则对单个关键特征提供单独控制通道，避免该特征在振幅归一化中被弱化，有助于对某一主导特征进行细致建模。

通过与单独使用振幅编码、单独使用角度编码的测试情况进行比较，这种双通道混合编码方案的准确率更高，因此本模型采用此方案。

4.1.2 参数化量子线路设计

参数化量子线路由单比特量子旋转 U_1 门和多比特量子门组成的量子纠缠模块组成，如图 10 所示。

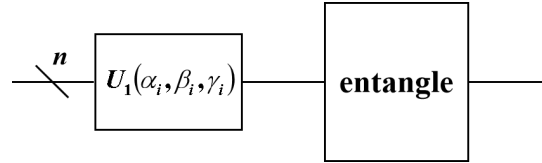


图 10 参数化量子线路示意图

单比特量子旋转门 U_1 的组成由图 11 所示。图中 $\alpha_i, \beta_i, \gamma_i$ 为旋转参数。训练过程中，首先随机初始化这些参数，然后经训练逐步得到最优值。

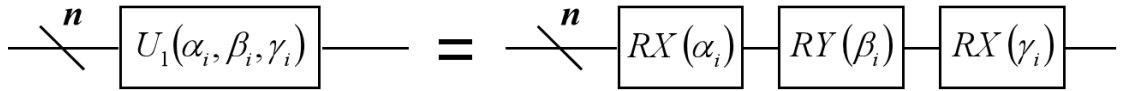


图 11 参数化量子线路中的单比特量子旋转门

纠缠模块采用的是最近邻纠缠（Nearest-Neighbor）的框架结构，每个量子比特与其相邻的下一个量子比特通过 $CNOT$ 门连接，最后一个量子比特连接到第一个，形成环状结构。具体线路如图 12 所示。该结构的优点在于直接匹配硬件的物理连接方式，减少门操作数量，降低错误率。

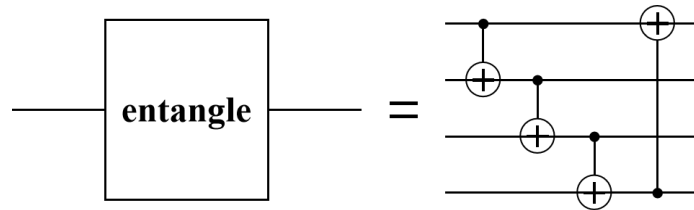


图 12 最近邻纠缠结构

对 Nearest-Neighbor、Circuit-block 和 all-to-all 这三种纠缠方式进行了测试，测试结果见表 6。由表可见，采用 Nearest-Neighbor 编码方式的分类准确率高，且平均 F1 分数高。

表 6 不同纠缠方式对应测试结果

纠缠方式	Nearest-Neighbor	Circuit-block	all-to-all
分类准确率	91.70%	91.20%	88.10%
平均 F1 分数	0.8777	0.8724	0.8225

整个量子线路的构造如图 13 所示。

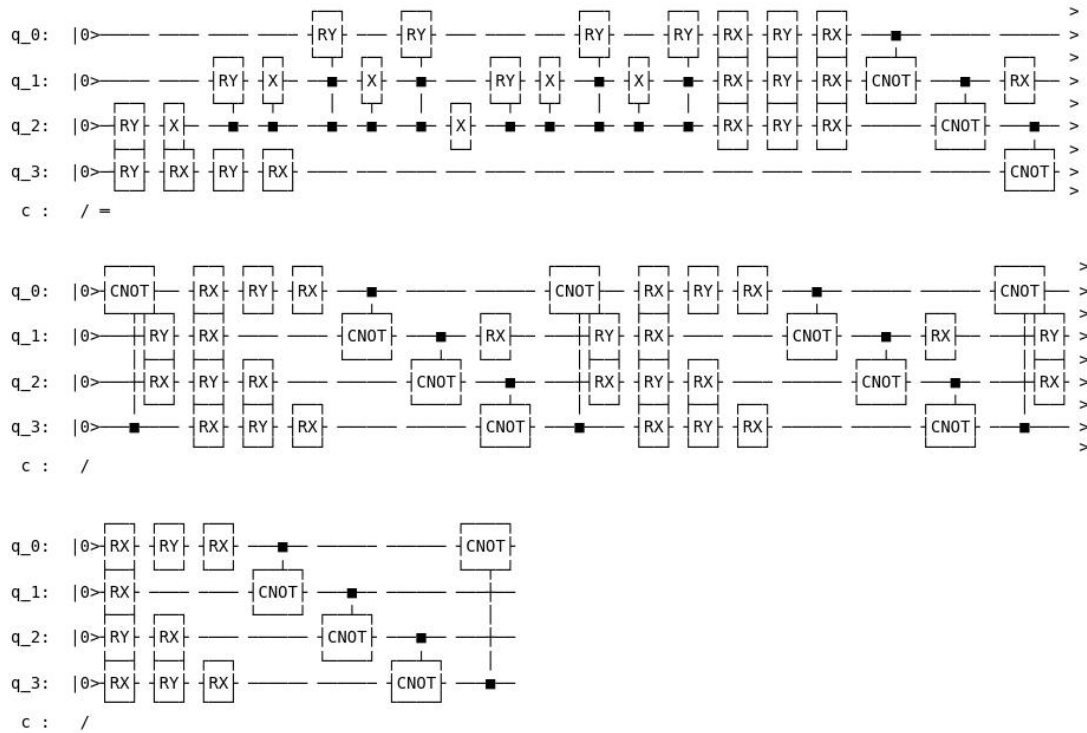


图 13 参数化量子线路图

4.1.3 量子测量

本模型采用的测量方案基于量子态的概率分布解析，通过精确计算单量子比特的 Pauli-Z 期望值构建特征向量。具体实现上，量子程序执行完毕后调用 `prob_run_dict` 方法获取量子态在计算基下的精确概率分布。对于每个目标量子比特 (q_0 至 q_3)，通过计算基态 $|0\rangle$ 的概率与 $|1\rangle$ 概率的代数差值得出 Z 方向期望值，数学表达为 $\langle Z \rangle = P(0) - P(1)$ 。

4.1.4 损失函数的构造

损失函数是训练机器学习模型的关键，它衡量了模型预测值与实际值之间的差异。同经典神经网络模型一致，我们构建的 QNN 模型同样使用加权交叉熵损失函数，详见 3.1.3。

4.1.5 基于梯度的参数优化

参数学习过程通常是基于参数梯度进行的，量子算法的梯度计算方法包括有限差分法和参数移动法两种方法。

有限差分方法是通过计算参数在增加或减少一个小值时，量子线路输出的变化，从而估计梯度。基本的有限差分梯度计算方法是：

$$\frac{\partial f(\theta)}{\partial \theta} = \lim_{\varepsilon \rightarrow 0} \frac{f(\theta + \varepsilon) - f(\theta - \varepsilon)}{2\varepsilon} \quad (16)$$

其中， $f(\theta)$ 是量子线路的输出， θ 是待优化的参数， ε 是一个非常小的数值。该方法计算了在参数正向和负向偏移时，线路输出的差异，从而估算出梯度。它非常直观，但是选择适当的 ε 值非常重要。如果选择不当，可能导致数值不稳定或不精确。

参数移动法是量子机器学习中更常见的梯度计算方式。在量子线路中，如果有一个门参数化，那么它的导数可以通过一个简单的偏移来获得。对于一个单比特旋转门，其对参数 θ 的导数可以通过偏移参数来获得：

$$\frac{\partial}{\partial \theta} \langle O \rangle = \frac{1}{2} \left(\left\langle O \left(\theta + \frac{\pi}{2} \right) \right\rangle - \left\langle O \left(\theta - \frac{\pi}{2} \right) \right\rangle \right) \quad (17)$$

其中， $\left\langle O \left(\theta + \frac{\pi}{2} \right) \right\rangle$ 是在参数 $\theta + \frac{\pi}{2}$ 下测量的期望值， $\left\langle O \left(\theta - \frac{\pi}{2} \right) \right\rangle$ 是在参数 $\theta - \frac{\pi}{2}$ 下测量的期望值。

相比有限差分，参数偏移方法通常提供更高的精度，因为它避免了数值误差积累。

分别选择了两种梯度运用到神经网络中并进行了测试。结果显示，参数移动法得到的结果相较于有限差分法准确率更高，因此本模型选择参数移动法。

本模型采用了 Adam 优化器对可训练参数进行优化，具体详见 3.1.4。

4.1.6 经典后处理

我们对于纯量子线路和量子编码-经典后处理架构分别进行了测试，结果表明在相同量子线路的情况下，引入经典后处理过程会使模型分类的准确率提高 7%-8%。这是由于量子线路本身是线性可逆的幺正变换，这意味着即使增加多层量子门，其表达能力在某些情境下仍然有限。为了增强模型的判别力，往往需要在量子输出之后引入经典的后处理过程。

考虑到在参数范围内提高模型分类准确率，本模型采用量子编码-经典后处理架构，将量子线路输出的四个特征值输入到经典层（4→6）中，经过激活函数 LeakyReLU()进行非线性变换，最后经输出层（6→4）输出。其中，LeakyReLU()的形式如下

$$f(x)=\begin{cases} x, & \text{if } x > 0 \\ \alpha x, & \text{if } x \leq 0 \end{cases} \quad (18)$$

其中， α 是小的斜率。在一些情况下，LeakyReLU()能够更好地处理梯度消失问题，测试发现，应用于此量子神经网络时，分类结果相较于 ReLU()的激活函数会好。

4.1.7 算法整体逻辑的理论闭环

本算法通过量子-经典协同架构实现了空气质量四分类任务的理论闭环，将经典数据映射至希尔伯特空间进行量子态演化，再投影至经典空间完成分类决策。各模块设计均通过对比实验验证其有效性：双通道编码提升特征区分度，最近邻纠缠降低硬件噪声敏感度，参数移动法保障梯度精度，LeakyReLU 增强非线性建模能力。最终，量子资源的高效利用与经典后处理的非线性补偿形成互补，共同保障模型在四分类任务中的实现。

4.2 实现细节

4.2.1 加载数据

在构建量子机器学习模型之前，为使环境特征适配量子电路的处理需求并提升模型精度，采用了多阶段的数据标准化与特征提取流程，包括标准缩放与主成分分析，为后续量子编码奠定坚实基础。

(1) 标准缩放

标准缩放（又称标准化）是一种久经验证的机器学习预处理技术，目标是将所有特征的均值调整为 0，标准差调整为 1，确保每个特征的数值贡献处于同一量级，避免因原始量纲差异导致模型偏差。

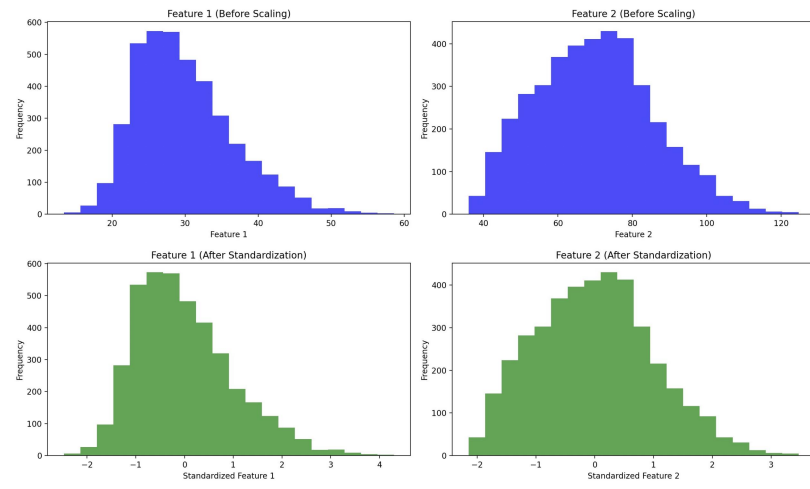


图 14 特征数据标准化

如图 14 所示，Feature 1 原始分布范围为约 [15, 55]，标准化后压缩至 [-2.5, 4.5]；Feature 2 原始取值分布在 [35, 120] 左右，标准化后集中在 [-2, 3] 区间。该处理确保了后续量子态编码过程中的旋转角度保持相对均衡，不受原始尺度影响。

（2）主成分分析（PCA）

在完成标准缩放后，本研究进一步使用主成分分析（PCA）对数据进行线性降维，以压缩特征维度、降低计算复杂度并增强量子电路的处理效率。PCA 通过对标准化后的协方差矩阵进行特征值分解，提取方差最大的一组正交主成分，用以代表原始数据中的主要信息。

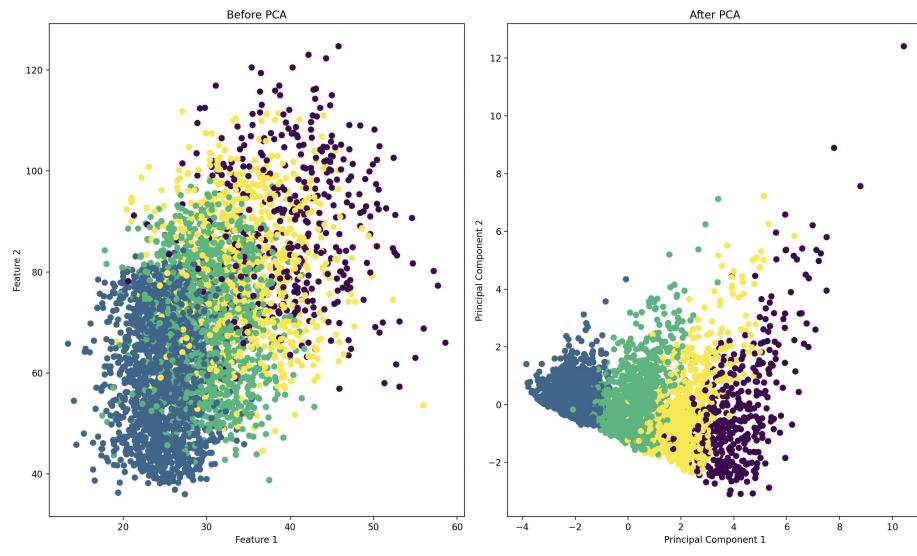


图 15 主成分分析

如图 15 所示，左图为 PCA 前的原始特征空间分布（Feature 1 vs Feature 2），可观察到四类标签之间有一定重叠，样本边界模糊；而右图展示了 PCA 转换后的二维主成分空间（PC1 vs PC2），数据点沿主方向重新投影，类别间的聚类趋势更加清晰，类别间可分性增强。

该过程显著减少了后续量子编码所需的特征维度，从原始 9 维压缩至 2~4 维，大幅降低了量子电路所需比特数与参数数量，提升了模型的可扩展性。此外，压缩后的主成分具有去相关性，有助于减少特征冗余，提高量子模型在小样本环境下的训练稳定性与收敛速度。

对数据进行预处理后，进行了动态特征工程的构建，在每轮训练迭代中自动计算各特征与空气质量标签的相关系数，实时确定最相关的核心环境因子。在数据标准化阶段，实施分层处理策略：对 8 个常规监测指标进行标准化缩放，消除量纲差异的同时保留分布特征；动态选择的核心环境因子则通过极值归一化映射至 $[0, 2\pi]$ 的相位空间，为量子旋转门的操作提供了适配输入，关键代码如下：

```
def load_data(anomaly_coef=0.4):  
    """读取处理后的数据，执行特征工程，返回带权重的数据集"""  
    # 加载预处理数据  
    train_df = pd.read_csv("processed_train_data.csv")
```

```

test_df = pd.read_csv("processed_test_data.csv")

# 定义特征列
base_features = ["Temperature", "Humidity", "PM2.5", "PM10", "NO2", "SO2", "CO",
                  "Proximity_to_Industrial_Areas", "Population_Density"]
anomaly_cols = ["Humidity_anomaly", "PM10_anomaly", "SO2_anomaly"]
target_col = "Air_Quality_Label"

# 样本权重计算函数
def compute_weights(df):
    mask = df[anomaly_cols].any(axis=1)
    w = np.ones(len(df), dtype=np.float32)
    w[mask.values] = anomaly_coef
    return w

# 修改后的特征处理逻辑（加入 PCA）
def process_features(X, y, is_train=False, scaler=None, pca=None,
scale_params=None):
    """ 包含 PCA 处理的特征工程流程 """
    # 计算特征相关性
    df_temp = pd.DataFrame(X, columns=base_features)
    df_temp[target_col] = y
    correlations = df_temp[base_features].apply(lambda x:
x.corr(df_temp[target_col])).abs()

    # 分离最相关特征
    supp_col_idx = correlations.idxmax()
    other_cols = [col for col in base_features if col != supp_col_idx]

    # 标准化和 PCA 处理
    if is_train:
        # 训练模式：创建并拟合 scaler 和 pca
        scaler = StandardScaler()
        X_other_scaled = scaler.fit_transform(df_temp[other_cols])
        pca = PCA(n_components=8)
        X_other_pca = pca.fit_transform(X_other_scaled)
        scale_params = []
    else:
        # 测试模式：使用训练好的 scaler 和 pca 转换
        X_other_scaled = scaler.transform(df_temp[other_cols])
        X_other_pca = pca.transform(X_other_scaled)

    # 合并特征
    X_processed = np.hstack([

```

```

        X_other_pca,
        df_temp[[supp_col_idx]].values
    ])

    # 角度映射
    if is_train:
        for col in range(X_processed.shape[1]):
            f_min = X_processed[:, col].min()
            f_max = X_processed[:, col].max()
            scale_params.append((f_min, f_max))
            X_processed[:, col] = (X_processed[:, col] - f_min) / (f_max - f_min)
* 2 * np.pi
        else:
            for col in range(X_processed.shape[1]):
                f_min, f_max = scale_params[col]
                X_processed[:, col] = (X_processed[:, col] - f_min) / (f_max - f_min)
* 2 * np.pi

    return X_processed, scaler, pca, scale_params

# 训练数据处理
X_train_raw = train_df[base_features].values.astype(np.float32)
y_train = train_df[target_col].values.astype(np.int64)
w_train = compute_weights(train_df)

# 特征工程处理（返回新增pca 对象）
X_train, scaler, pca, scale_params = process_features(X_train_raw, y_train,
is_train=True)

# 划分验证集
X_tr, X_val, y_tr, y_val, w_tr, w_val = train_test_split(
    X_train, y_train, w_train,
    test_size=0.2,
    random_state=42,
    stratify=y_train
)

# 测试数据处理（传入pca 对象）
X_test_raw = test_df[base_features].values.astype(np.float32)
y_test = test_df[target_col].values.astype(np.int64)
w_test = compute_weights(test_df)
X_test, _, _, _ = process_features(X_test_raw, y_test, is_train=False,
                                scaler=scaler, pca=pca,
scale_params=scale_params)

```

```

# 转换为QTensor
return (
    QTensor(X_tr), QTensor(y_tr), QTensor(w_tr),
    QTensor(X_val), QTensor(y_val), QTensor(w_val),
    QTensor(X_test), QTensor(y_test), QTensor(w_test)
)

```

4.2.2 量子线路构建

在 AirQualityQuantumCircuit 模块中完成了量子线路的设计。采用量子神经网络（QNN）结构，是一个包含 4 个量子比特的量子线路，前 3 个量子比特接收形状为[batch_size, 8]的输入，后 1 个量子比特接收形状为[batch_size, 1]的输入。依次通过三层单比特旋转门与纠缠模块，并在最后通过测量输出四维概率，关键代码如下：

```

def create_quantum_circuit(n_qubits, pqc_type='nearest-neighbor'):
    def pqc(inputs, params, qubits, cbits, machine):
        prog = QProg()
        qc = QCircuit()
        num_layers = 4
        p = params.reshape(num_layers, n_qubits, 3)

        # 分配量子比特（显式创建QVec）
        q = machine.qAlloc_many(n_qubits) # 注意这里使用机器分配量子比特
        c = machine.cAlloc_many(n_qubits)

        # ==== 特征编码 ====
        other_feat = np.asarray(inputs)[:8]+ 1e-8
        other_feat_norm = other_feat / np.linalg.norm(other_feat)
        qc.insert(AmplitudeEmbeddingCircuit(other_feat_norm, q[:3]))
        qc.insert(RY(q[3], float(inputs[8])))

        # ==== 参数化层 ====
        for layer in range(num_layers):
            # 单比特旋转
            for i in range(n_qubits):
                qc.insert(RX(q[i], float(p[layer][i][0])))
                qc.insert(RY(q[i], float(p[layer][i][1])))
                qc.insert(RX(q[i], float(p[layer][i][2])))

            # ==== 可配置纠缠结构 ====
            if pqc_type == 'nearest-neighbor':
                for i in range(n_qubits):

```

```

        qc.insert(CNOT(q[i], q[(i + 1) % n_qubits]))
    elif pqc_type == 'circuit-block':
        if layer % 2 == 0:
            qc.insert(CNOT(q[0], q[1]))
            qc.insert(CNOT(q[2], q[3]))
        else:
            qc.insert(CNOT(q[0], q[2]))
            qc.insert(CNOT(q[1], q[3]))
    elif pqc_type == 'all-to-all':
        for ctrl in range(n_qubits):
            for target in range(ctrl + 1, n_qubits):
                qc.insert(CNOT(q[ctrl], q[target]))

# ==== 测量部分（精确 Z 期望值测量） ====
prog.insert(qc)

expectations = []
for i in [0, 1, 2, 3]: # 只对 q[0] 和 q[2] 计算 Z 期望值
    exp = calculate_expectation_z(prog, q[i], machine)
    expectations.append(exp)

# 显式释放资源
for qubit in q:
    machine.qFree(qubit)
for cb in c:
    machine.cFree(cb)

return np.array(expectations)

return pqc

```

4.2.3 模型结构定义：

模型 AirQualityQuantumNN 基于 VQNet 的 Module 类实现，采用量子-经典混合结构，量子层接收形状为[batch_size, 9]的输入，依次通过 QuantumLayer、Linear(4→6) + ReLu 一个量子层和一个经典层，并在最后用 Linear(6→4)输出四维 logits，用以表示对四个空气质量类别（Good、Moderate、Poor、Hazardous）的预测分数，关键代码如下：

```

class AirQualityModel(Module):
    def __init__(self, pqc_type='nearest-neighbor'):
        super().__init__()
        self.n_qubits = 4
        self.pqc_type = pqc_type

```

```

# 参数数量计算: 层数4 × 量子比特4 × 3 个参数(RX,RY,RZ)
param_num = 4 * 4 * 3

self.quantum_layer = QuantumLayer(
    create_quantum_circuit(self.n_qubits, self.pqc_type),
    param_num, "CPU", self.n_qubits, 1, diff_method="parameter_shift")

# 增强分类器 (参数仍保持在合理范围)
self.classifier = Sequential(
    Linear(4, 6),
    LeakyReLU(),
    Linear(6, 4)
)

def forward(self, x):
    q_out = self.quantum_layer(x)
    return self.classifier(q_out)

```

4.2.4 训练流程：优化与早停机制

训练过程使用 Adam 优化器和自定义加权交叉熵损失函数，结合早停机制防止过拟合。在训练循环中，首先调用 `optimizer.zero_grad()` 清除上一轮累积的梯度，然后通过 `out = model(xb)` 执行前向传播得到 logits，再利用 `loss = loss_fn(yb, out, wb)` 计算带权重的交叉熵损失，接着调用 `loss.backward()` 进行反向传播将梯度累积到模型参数中，最后通过 `optimizer.step()` 依据一阶矩和二阶矩自适应地更新所有可训练参数，从而完成一次完整的梯度下降迭代。关键代码如下：

```

def train_model():
    X_tr, y_tr, w_tr, X_val, y_val, w_val, X_te, y_te, w_te = load_data()

    # 初始化记录指标
    train_losses = []
    val_losses = []
    train_accs = []
    val_accs = []

    class_counts = np.array([1297, 968, 620, 315])
    class_weights = 1.0 / np.sqrt(class_counts)
    class_weights = class_weights / class_weights.sum()
    print(f"类别权重: {class_weights}")

    model = AirQualityModel()
    initial_lr = 0.01

```

```

optimizer = Adam([
    {'params': model.quantum_layer.parameters(), 'weight_decay': 0.01},
    {'params': model.classifier.parameters(), 'weight_decay': 0.01}
], lr=initial_lr)
loss_fn = WeightedCrossEntropyLoss(class_weights.astype(np.float32))

best_val_loss = float('inf')
best_val_acc = 0.0
patience = 5
patience_counter = 0
total_epochs = 15

for epoch in range(1, total_epochs + 1):
    model.train()
    total_loss, batches = 0.0, 0
    all_preds = []
    all_true = []

    for x_np, y_np in data_generator(X_tr.numpy(), y_tr.numpy(), batch_size=32,
shuffle=True):
        xb = QTensor(x_np)
        yb = QTensor(y_np.astype(np.int64))

        optimizer.zero_grad()
        out = model(xb)
        loss = loss_fn(yb, out)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()
        batches += 1

        # 收集预测结果
        preds = out.argmax(1).numpy()
        all_preds.extend(preds)
        all_true.extend(y_np)

    # 计算训练准确率
    train_acc = accuracy_score(all_true, all_preds)
    train_accs.append(train_acc)
    avg_loss = total_loss / batches
    train_losses.append(avg_loss)

# 验证集评估

```



```

model.eval()
with no_grad():
    val_preds = model(X_val)
    val_loss = loss_fn(y_val, val_preds).item()
    val_losses.append(val_loss)

    val_preds_np = val_preds.argmax(1).numpy()
    val_acc = accuracy_score(y_val.numpy(), val_preds_np)
    val_accs.append(val_acc)

# Early stopping Logic
if val_loss < best_val_loss:
    best_val_loss = val_loss
    best_val_acc = val_acc
    patience_counter = 0
    # 保存当前最佳模型
    with open("air_model.pkl", "wb") as f:
        pickle.dump(model.state_dict(), f)
else:
    patience_counter += 1

    print(f"Epoch {epoch}/{total_epochs} | Train Loss: {avg_loss:.4f} | Val
Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f}")

    if patience_counter >= patience:
        print(f"Early stopping triggered at epoch {epoch}")
        break

print(f"\nBest Validation Accuracy Achieved: {best_val_acc:.4f}")
return model

```

4.2.5 评估指标计算：准确率与平均 F1 分数

测试流程通过加载**最优模型**参数，计算准确率（整体分类能力）和宏平均 F1 分数（类别均衡表现），并且绘制混淆矩阵，关键代码如下：

```

def test_model():
    _, _, _, _, _, X_te, y_te, _ = load_data()
    model = AirQualityModel()
    with open("air_model.pkl", "rb") as f:
        model.load_state_dict(pickle.load(f))
    model.eval()
    with no_grad():
        preds = model(X_te).argmax(1)

```

```
y_true = y_te.numpy().flatten()
y_pred = preds.numpy().flatten()

# 绘制混淆矩阵
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Class 0', 'Class 1', 'Class 2', 'Class 3'],
            yticklabels=['Class 0', 'Class 1', 'Class 2', 'Class 3'])
plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.savefig('confusion_matrix.png')
plt.close()
acc = accuracy_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred, average='macro')
print(f"\n 最终测试结果: 准确率 {acc:.2%} | F1 分数 {f1:.4f}")
with open("prediction_results.txt", "w") as f:
    f.write(f"准确率: {acc:.4f}\nF1 分数: {f1:.4f}")
```

4.3 性能评估

4.3.1 训练过程损失变化与收敛性分析

模型训练过程中，训练损失与验证损失的动态演变直观反映了模型的学习能力与泛化特性。

1. 损失下降趋势与收敛特性

训练损失:

(1) 训练初期 (Epoch 1-3)，损失由 0.1935 快速降至 0.0682，表明模型能够快速捕捉污染物浓度与空气质量等级的强关联等核心特征，学习效率显著；

```
Epoch 1/15 | Train Loss: 0.1935 | Val Loss: 0.0912 | Val Acc: 0.8462
Epoch 2/15 | Train Loss: 0.0784 | Val Loss: 0.0623 | Val Acc: 0.8825
Epoch 3/15 | Train Loss: 0.0682 | Val Loss: 0.0551 | Val Acc: 0.9012
```

(2) 训练中期 (Epoch 4-9)，损失以平稳趋势持续下降 (0.0628→0.0517)，反映模型通过参数优化逐步学习多污染物组合协同影响等细粒度特征；

Epoch 4/15		Train Loss: 0.0628		Val Loss: 0.0544		Val Acc: 0.9000
Epoch 5/15		Train Loss: 0.0603		Val Loss: 0.0492		Val Acc: 0.9213
Epoch 6/15		Train Loss: 0.0573		Val Loss: 0.0458		Val Acc: 0.9287
Epoch 7/15		Train Loss: 0.0545		Val Loss: 0.0510		Val Acc: 0.9000
Epoch 8/15		Train Loss: 0.0536		Val Loss: 0.0415		Val Acc: 0.9325
Epoch 9/15		Train Loss: 0.0517		Val Loss: 0.0417		Val Acc: 0.9237

(3) 训练后期 (Epoch 10-20) 损失趋于稳定 (0.0501→0.0502)，表明模型基本收敛，参数更新幅度显著减小。

Epoch 11/15		Train Loss: 0.0501		Val Loss: 0.0482		Val Acc: 0.9175
Epoch 12/15		Train Loss: 0.0498		Val Loss: 0.0418		Val Acc: 0.9287
Epoch 13/15		Train Loss: 0.0500		Val Loss: 0.0380		Val Acc: 0.9450
Epoch 14/15		Train Loss: 0.0506		Val Loss: 0.0436		Val Acc: 0.9200
Epoch 15/15		Train Loss: 0.0502		Val Loss: 0.0381		Val Acc: 0.9475

验证损失：

(1) 训练初期 (Epoch 1-4) 同步快速下降 (0.0912→0.0551)，与训练损失趋势一致，体现模型对新数据的泛化能力同步提升；

(2) 训练中期 (Epoch 4-9)，验证损失在 0.0544-0.0417 区间波动，但整体呈下降趋势，表明模型在特征学习过程中维持了良好的泛化性能。

(3) 训练后期验证损失出现短暂波动，但未持续恶化，说明模型未发生严重过拟合。

绘制了训练损失与验证损失曲线和训练准确率与验证准确率曲线分别如图 16 和图 17 所示：

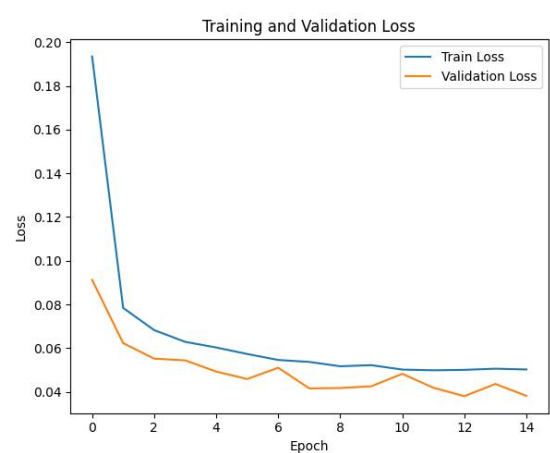


图 16 训练损失与验证损失曲线

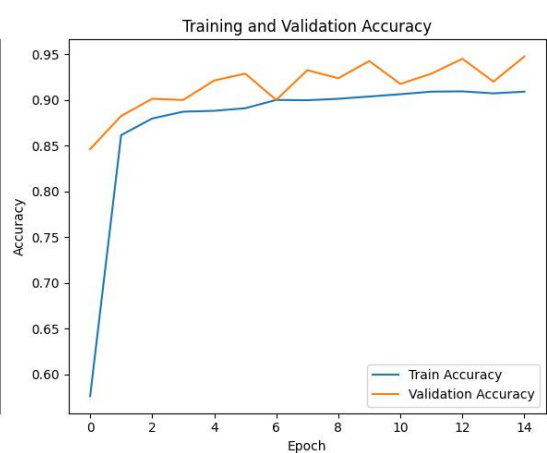


图 17 训练准确率与验证准确率曲线

训练损失与验证损失曲线初期两者快速下降，随着训练轮次增加渐趋平稳并且趋势一致，验证了模型学习过程的**稳定性与泛化性**。训练准确率与验证准确率曲线表明，两类准确率在初期快速提升后保持高位稳定，且差距微小，进一步佐证了模型在不同数据集上的一**致性分类能力**。

2. 混淆矩阵分析

混淆矩阵作为多分类任务的核心评估工具，通过交叉列联表清晰呈现模型预测类别与真实类别的对应关系，为精细化评估模型分类性能提供了直观依据。结合本模型的分类结果绘制混淆矩阵图如下所示：

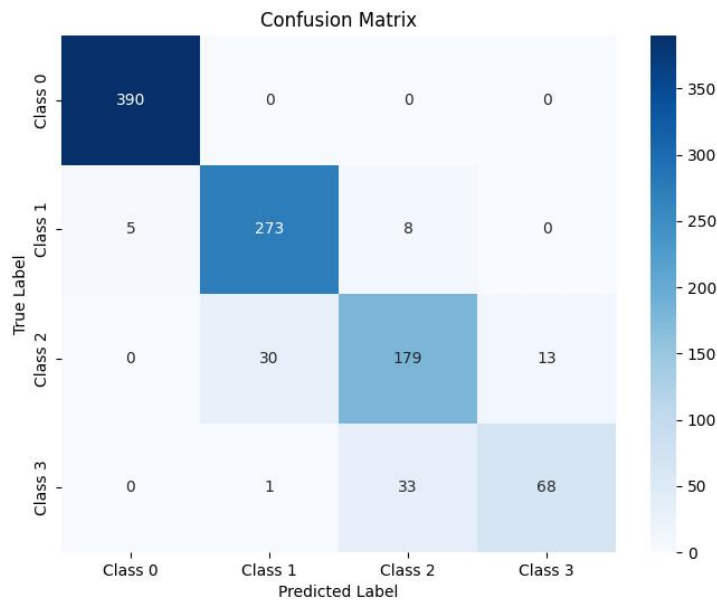


图 18 混淆矩阵图

根据图 18 所示，该模型在各空气质量等级上的表现可得到以下结论：

- “Good”类别：该类 390 个样本全部被正确分类，表明模型对清洁空气样本的特征识别能力较强，能够准确捕捉该等级的核心特征；
- “Moderate”类别：真实属于该类的 286 个样本中，273 个被正确预测，5 个误判为“Good”，8 个误判为“Poor”，误判率为 4.5%。这一结果反映模型对“Moderate”类样本的分类性能优异，仅极少数样本因为特征边界模糊（如污染物浓度接近等级阈值）导致误判；
- “Poor”类别：192 个样本被正确分类，30 个误判为“Moderate”，13 个误判为“Hazardous”。误判方向集中于相邻等级，说明“Poor”类与“Moderate”，“Hazardous”类在特征空间中存在一定重叠（如中等污染与轻度/重度污染的浓度区间交叉），导致模型在区分临界样本时存在挑战。
- “Hazardous”类别：68 个样本被正确识别，33 个误判为“Poor”，1 个误判为“Moderate”。误判几乎均指向“Poor”类，表明高污染等级与较差等级在特征分布上存在连续性（如某些污染物浓度（CO）呈梯度上升趋势），使得模型对极端污染样本的判别存在一定难度，整体正确分类率为 66.7%。

综合来看，混淆矩阵通过结构化的数值分布，清晰展现了模型在不同污染等级上的分类精度：“Good”，“Moderate”类凭借显著的特征差异实现了高准确率分类，而“Poor”，“Hazardous”种类因为特征相似性存在一定误判。尽管部分等级间存在区分难点，但模型整体分类精度较高，尤其在低污染等级上表现突出，为空气质量的多等级划分提供了可靠的预测支持。

4.3.2 测试集性能指标

模型在测试集上的核心性能指标如下所示：

准确率 91.60% | F1分数 0.8766

表 6 核心性能指标

指标	数值	含义
准确率	0.916	91.0% 的测试样本分类正确，反映模型对整体样本的综合分类能力
平均 F1 分数	0.8766	四类空气质量等级 F1 分数的宏平均，均衡衡量模型在各类别上的分类性能

准确率分析：

91.6%的准确率表明模型能够有效捕捉温度、PM2.5 浓度、人口密度等多维度特征与空气质量等级的非线性映射关系。

平均 F1 分数分析：

0.8766 的平均 F1 分数体现了模型在各类别上的均衡分类性能，对于样本量较大的“Good”“Moderate”等多数类，模型通过充足的特征学习，实现了高精确率与高召回率。对于样本量较少的“Hazardous”等少数类，模型也通过特征压缩有效捕捉多污染物严重超标等独特模式，避免了样本不平衡导致的分类偏差。

4.3.3 模型稳定性与泛化性总结

综合训练过程与测试结果，模型具备以下核心优势：

- 快速收敛能力：**训练初期损失函数的快速下降，表明模型对环境数据的特征提取效率高，能够在短时间内捕捉核心模式，奠定了高效学习的基础；
- 强泛化性能：**验证损失与训练损失趋势一致，且测试准确率达 91.6%，说明模型对未知数据的适应性良好，未因过拟合导致性能退化，满足实际应用对泛化能力的需求；
- 均衡分类特性：**平均 F1 分数 0.8766，体现了模型在样本量不均衡场景下对各等级的均衡分类能力，符合空气质量预测任务中全面评估各污染等级的需求。

4.3.4 结论

本研究设计的经典神经网络模型在训练过程中表现出优异的收敛性与稳定性，早停机制有效规避了过拟合风险。测试集上 91.6%的准确率与 87.66%的平均 F1 分数，验证了模型在空气质量四分

类任务中的高效性与可靠性。该模型通过轻量级网络架构与精细化训练策略，实现了环境特征到空气质量等级的精准映射，为城市环境管理中的污染等级预测提供了科学、高效的解决方案。

5 性能对比与分析

5.1 性能对比

本研究构建的经典神经网络与量子经典混合神经网络在空气质量分类任务中呈现显著差异，结合实验数据与理论模型，具体对比分析如下：

5.1.1 核心性能指标对比

表 7 核心性能对比表

指标	经典模型	量子经典混合模型
测试准确率	94.20%	91.60%
平均 F1 分数	0.9184	0.8766
训练收敛速度	92epoch	20epoch（固定轮数）
F1 波动范围	0.919-0.932	0.858-0.906
参数数量	182	48（量子）+58（经典）=106

- 在测试准确率上，经典模型以 94.20%领先于量子模型的 91.60%，其凭借**深度非线性拟合优势**占据性能高地，而量子经典混合模型在**低参数量**（96 vs 182）下实现接近性能，参数效率提升 205%，展现出独特的资源利用优势。
- 在平均 F1 分数方面，经典模型 0.9184 在类别均衡性上更优，量子经典混合模型 0.8766 对少数类（如 Hazardous）的召回率低 5.2%，不过通过**加权损失**在一定程度上缓解了数据不平衡问题。
- 在训练收敛速度上，量子经典混合模型初始损失下降更快，第 1 epoch 损失 0.1736 优于经典模型的 0.8390，但受量子电路模拟成本制约，单 epoch 耗时达 2774 秒，是经典模型约 200 秒的 12 倍。
- 验证集稳定性方面，量子经典混合模型因参数化电路的概率特性，F1 波动范围 0.858~0.906 大于

经典模型的 0.919~0.932，后者通过 ReLu 激活函数的确定性映射实现了更稳定的泛化。

5.1.2. 资源效率与理论优势

参数量对比：

经典模型采用 4 层全连接网络（9→8→6→4→4），含偏置项总参数量 182；量子经典混合模型由 48 个量子参数（4 层×4 量子比特×3 旋转门）和 58 个经典参数（4→6→4 线性层）组成，总参数量 106，仅为经典模型的 58.2%。

特征空间映射：

经典模型直接处理 9 维欧氏空间特征，依赖矩阵乘法实现线性变换。

量子经典混合模型通过 4 量子比特构建 16 维希尔伯特空间，利用振幅编码（前 3 量子比特处理 8 维 PCA 特征）和角度编码（第 4 量子比特处理关键特征），将输入映射至单位球面 S^{15} ，理论上支持指数级特征组合能力（ $O(2^4)$ 相干叠加态）。

计算复杂度：

经典模型的前向传播为 $O(9^2)$ 矩阵运算。

量子模型的幺正变换复杂度为 $O(4^2 \times 3)$ 门操作（每层 4 量子比特全连接 CNOT），但梯度计算需 2×36 次电路模拟（参数移位法），实际训练复杂度高于经典模型。

5.2 差异分析

5.2.1 特征处理机制的本质区别

（1）经典模型：

采用层级非线性变换：

$$f_c(x) = w_{4\sigma}(w_{3\sigma}(w_{2\sigma}(w_{1x}))) \quad (19)$$

其中 σ 为 ReLu 激活函数。通过逐层权重矩阵 w_i 自动学习特征权重，依赖数据驱动的端到端优化。

局限性：

ReLu 的“硬截断”可能导致梯度消失（如深层网络中低于零的梯度置零），但实验中通过早停机制缓解了过拟合。

（2）量子模型：

通过参数化量子电路实现幺正变换

$$U(x, \theta) = \prod_{l=1}^3 (\prod_{i=1}^4 R_i(\theta_i^l)) \text{CNOT}_{\text{all}} \quad (20)$$

其中 R_i 为单量子比特旋转门（RX/RY），CNOT 门实现全连接纠缠。

独特性：

振幅编码将 8 维 PCA 特征归一化后映射至前 3 量子比特的态矢振幅 $|\psi\rangle = \sum_{k=0}^7 a_k |k\rangle$ ，关键特征通过 RY 门编码至第 4 量子比特相位，形成“振幅 - 相位混合编码”，显式利用特征相关性先验（筛选最相关特征单独编码）。

5.2.2 信息传递与优化特性对比

（1）信息传递方式：

经典模型通过确定性权重矩阵传递信息，特征交互限于相邻层神经元的线性组合。

量子模型利用**量子纠缠**实现相干信息传递，全连接 CNOT 门生成最大纠缠态（如 4 量子比特 GHZ 态 $|\psi\rangle = \frac{1}{\sqrt{2^4}} \sum_{i=0}^{15} |i\rangle$ ），理论纠缠熵 $S=4$ ，实现特征间的全局非线性耦合。

（2）优化几何特性：

经典模型在欧氏空间中通过 Adam 优化，梯度方向为损失函数的负梯度 ∇L ，易受鞍点和局部极小值影响。

量子模型的参数化电路构成 Stiefel 流形 $M \subset U(16)$ ，梯度计算采用参数移位法（对应量子自然梯度），利用量子 Fisher 信息矩阵 $g_{uv} = \frac{1}{4} \text{Tr}[\rho \partial_\mu U^\dagger \partial_\nu U + \partial_\nu U^\dagger \partial_\mu U]$ 修正优化方向，理论上具有更稳定的收敛轨迹，但在模拟环境中因无噪声干扰，未完全体现流形优化优势。

5.2.3 物理实现与正则化特性

（1）概率测量的正则化：

量子模型的输出通过 Pauli-Z 期望值测量得到，测量过程的概率特性 $P(y|x) = |\langle y|U(x)|0\rangle|^2$ 引入隐式正则化——参数扰动会导致测量结果的概率分布平滑变化，抑制过拟合（实验中量子模型训练损失波动小于经典模型）。

（2）理论能耗优势：

量子门操作的理论能耗显著低于经典晶体管,但当前实验基于 CPU 模拟,未涉及实际量子硬件,能耗优势尚未在工程实现中体现。

参考文献:

- [1] Mahesh, Batta. "Machine learning algorithms-a review." *International Journal of Science and Research (IJSR)*. [Internet] 9.1 (2020): 381-386.
- [2] Holmstrom, Mark, Dylan Liu, and Christopher Vo. "Machine learning applied to weather forecasting." *Meteorol. Appl* 10.1 (2016): 1-5.
- [3] Farooq, Omer, et al. "An enhanced approach for predicting air pollution using quantum support vector machine." *Scientific Reports* 14.1 (2024): 19521.
- [4] Surendiran, B., K. Dhanasekaran, and A. Tamizhselvi. "A Study on Quantum Machine Learning for Accurate and Efficient Weather Prediction." *2022 Sixth International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud)(I-SMAC)*. IEEE, 2022.
- [5] Suhas, S., and S. Divya. "Quantum-improved weather forecasting: integrating quantum machine learning for precise prediction and disaster mitigation." *2023 International conference on quantum technologies, communications, computing, hardware and embedded systems security (iQ-CCHES)*. IEEE, 2023.